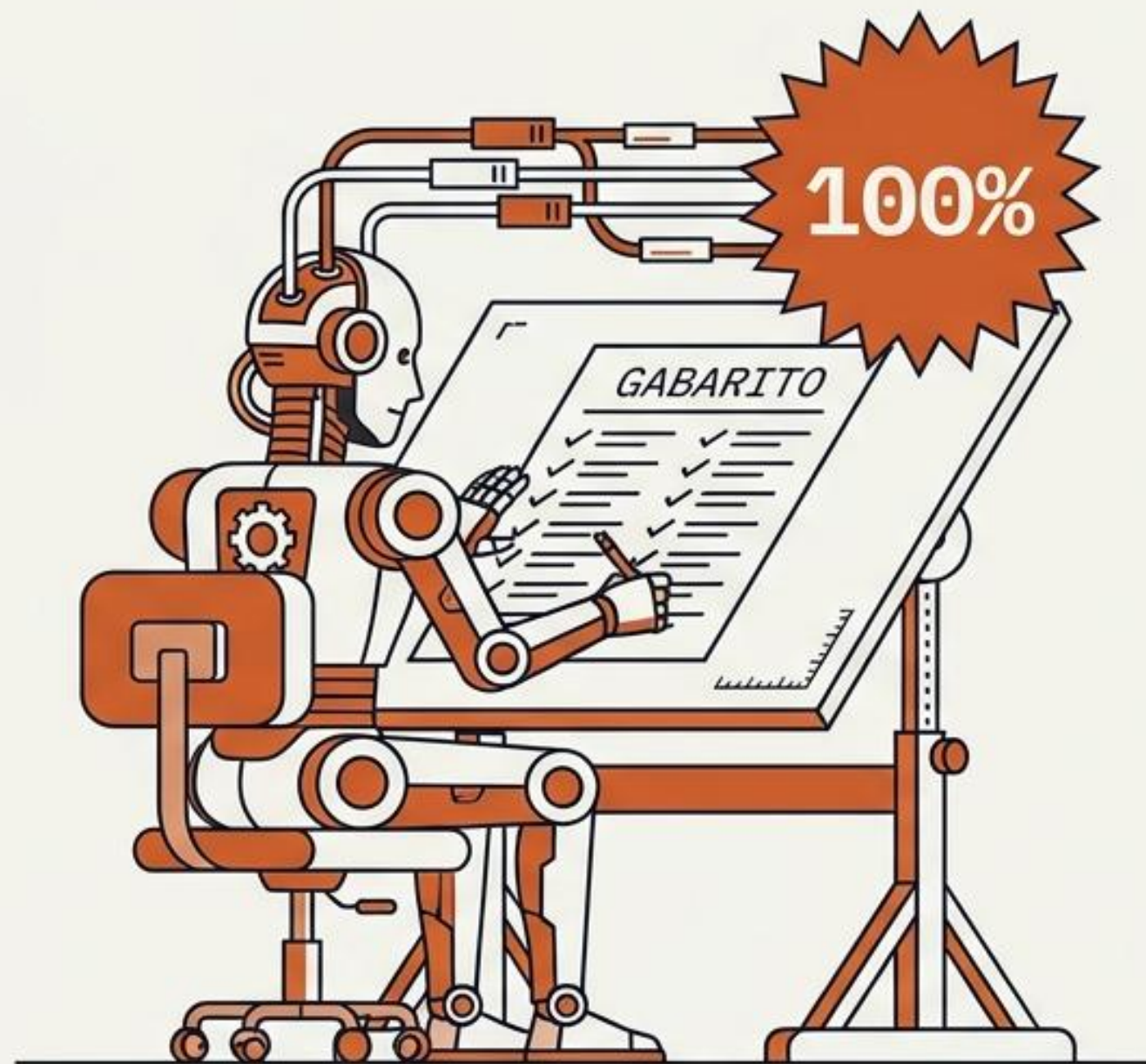


Playbook de Validação

A Ilusão da Acurácia

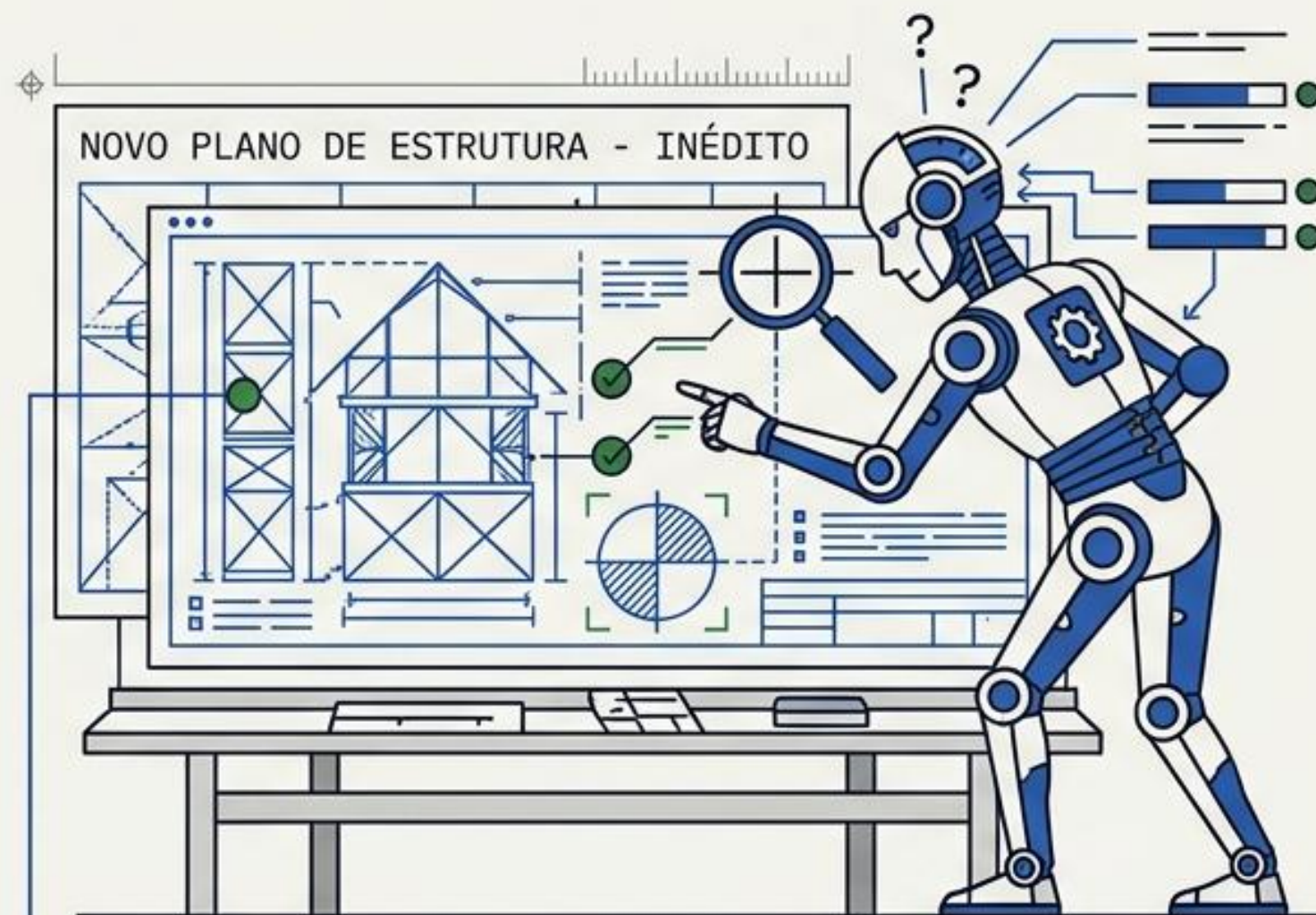
Como avaliar modelos de Machine Learning com honestidade, evitar vazamento de dados e garantir generalização real.

A Ilusão



Um modelo não é útil por acertar o que já viu.

A Realidade



O **"Erro de Generalização"** é a única métrica que importa no mundo real. A capacidade de acertar dados novos. **Avaliar é simular o futuro.**

Treino



Ajustar **parâmetros**
(coeficientes, pesos).

Uso: Quantas vezes o
algoritmo precisar.

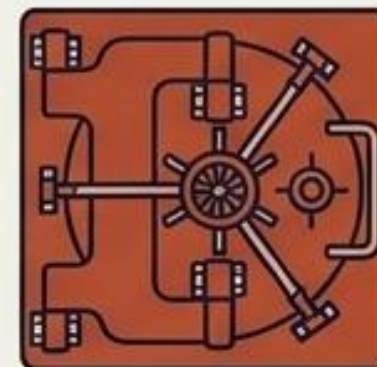
Validação



Escolher **hiperparâmetros**
e **modelos** (comparar
algoritmos).

Uso: **Várias vezes** durante
o desenvolvimento.

Teste

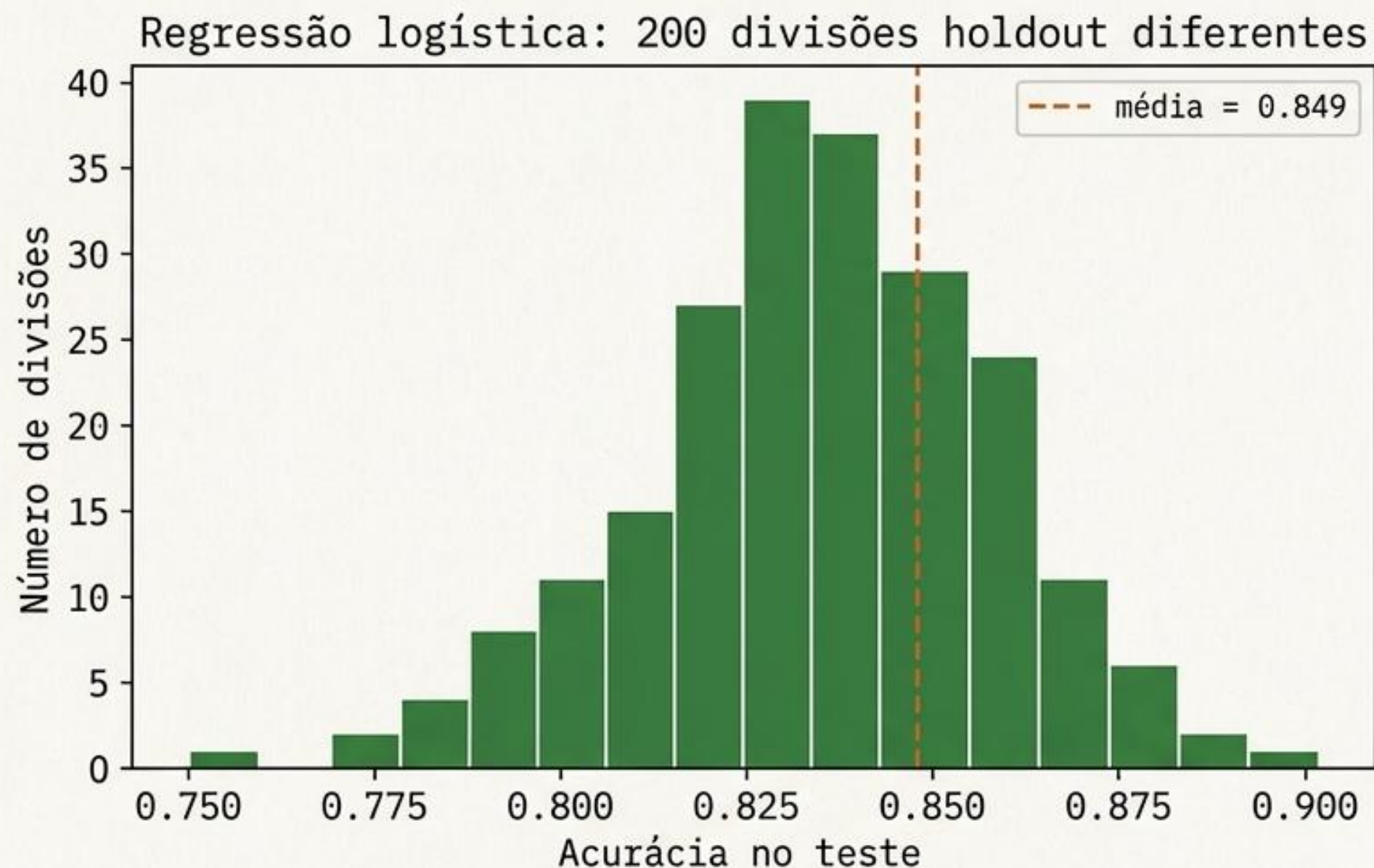


A **estimativa final**
e honesta.

Uso: Uma única vez.

A REGRA DE OURO: O conjunto de teste **NÃO PODE** influenciar decisões. Se
você o usar para escolher um **hiperparâmetro**, sua nota virou **otimista**.

A Armadilha do Holdout: A Sorte do Sorteio



Dividir os dados apenas uma vez (ex: 30/70) é rápido, mas perigoso.

A mesma máquina, nos mesmos dados, produziu resultados entre 74% e 91% dependendo apenas de quais exemplos caíram no teste.

Uma divisão única oculta a verdadeira incerteza do modelo. Quem faz um único sorteio pode reportar qualquer nota.

Uso correto de Treino, Validação e Teste

O conjunto de teste é usado apenas uma única vez, no final!

O conjunto de teste
NÃO PODE influenciar
decisões.



**Dados
totais**

(ex.: 1000 registros)

80%
Conjunto de Treino
(usado para treinar e validar)
800 registros

20%
Conjunto de Teste
(usado apenas uma vez, no final)
200 registros

1

Treino + Validação

Escolha dos hiperparâmetros

Usamos apenas o conjunto de treino
(e validação cruzada) para testar
diferentes hiperparâmetros.

Validação cruzada (K-Fold)

Treino
(800 registros)



■ Treino ■ Validação

Testamos
vários valores
de K:

K = 1
K = 3
K = 5
K = 7
K = 9

2

Escolha do modelo

Defina o melhor hiperparâmetro

Com base no desempenho médio
da validação cruzada, escolhemos
o melhor valor.

K	Acurácia média (validação cruzada)
1	0,82
3	0,86
5	0,90
7	0,87
9	0,84



Escolhemos K = 5
(melhor desempenho na validação)

3

Teste (apenas uma vez)

Avaliação final

Treinamos o modelo final com todo
o conjunto de treino (800 registros)
usando o K escolhido e avaliamos
no conjunto de teste.

Modelo final

KNN (K = 5)
treinado com 800 registros

Conjunto de teste
200 registros

Resultado final

Acurácia: 0,89
(exemplo)



Forma correta

- Use o teste apenas uma única vez, no final.
- Escolha hiperparâmetros usando apenas o conjunto de treino (validação cruzada).
- O teste fornece uma estimativa realista do desempenho em dados novos.
- Mantenha o conjunto de teste separado durante todo o processo.



Forma incorreta

- Usar o conjunto de teste para escolher hiperparâmetros.
- Testar várias opções e escolher a que deu o melhor resultado no teste.
- Olhar o resultado do teste, ajustar o modelo e testar novamente.
- Isso faz o teste virar uma espécie de validação e a nota fica otimista (maior do que a realidade).

O teste responde apenas a uma pergunta:

“Como o modelo escolhido se comporta em dados que ele nunca viu?”

Guarde o teste até o final!

A Solução: Validação Cruzada (K-Fold)



Embaralha os dados, divide em K dobras. Treina em K-1 dobras, valida na restante. Repete K vezes para que todos os dados sejam testados.

O resultado não é mais uma nota de sorte, mas um espectro estável: **Média ± Desvio Padrão** (Ex: **0,849 ± 0,030**).

O que o **cross_val_score** faz essencialmente?

Ele executa a validação cruzada: divide os dados, treina o modelo em cada dobra, avalia no conjunto de validação e retorna os scores de todas as dobras.

1 Dados de entrada

X (60 × 2000)

	f1	f2	f3	...	f2000
1	...	...	...	...	...
2	...	...	...	...	...
3	...	...	...	...	...
...	...	...	...	...	...
60	...	...	...	...	...

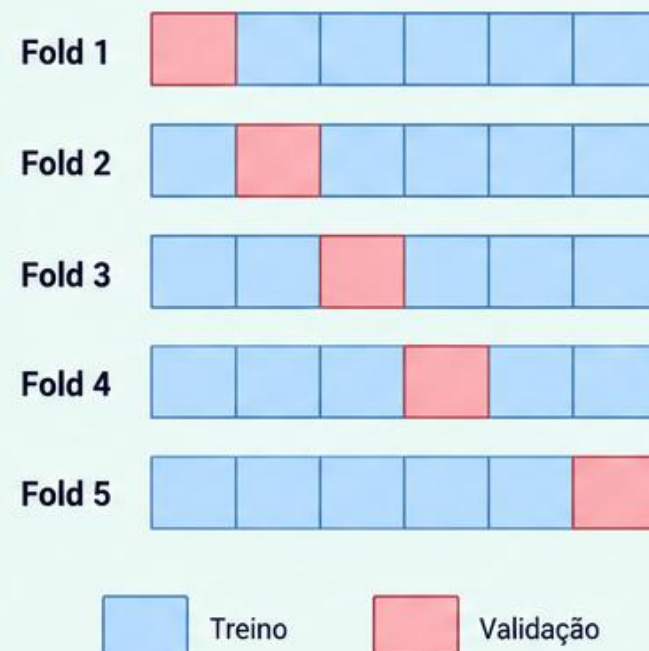
y (60)

1
0
1
...
0

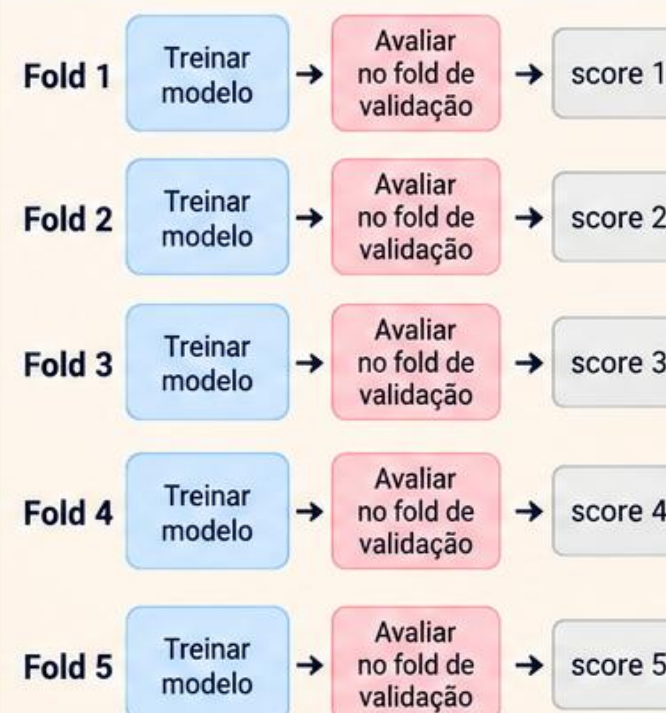
X e y são os dados completos que serão divididos em k dobras.

2 Divide os dados (K-Fold)

Usa o objeto de divisão (ex.: `StratifiedKFold`) para gerar k dobras. Exemplo: k = 5



3 Para cada dobra, faz:



4 Retorna os resultados

`scores = cross_val_score(...)`

Um array com o score de cada dobra:

`[s1, s2, s3, s4, s5]`

Exemplo:

`scores = [0.83, 0.75, 0.92, 0.83, 0.75]`

Normalmente usamos a média:

`scores.mean()`

Média do desempenho do modelo na validação cruzada.

Resumo do processo

- 1 Divide os dados em k dobras.
- 2 Para cada dobra: treina o modelo com k-1 partes e avalia na parte restante.
- 3 Coleta o score de cada dobra.
- 4 Retorna um array com os k scores (e geralmente usamos a média).

Exemplo de uso no seu código

```
dobras = StratifiedKFold(5, shuffle=True, random_state=SEMENTE)
scores = cross_val_score(modelo, X, y, cv=dobras)
print(scores)
print(scores.mean())
```

Aqui, o `cross_val_score` faz todo o processo de validação cruzada usando as dobras definidas pelo `StratifiedKFold`.

Em resumo: `cross_val_score` automatiza a validação cruzada, executando o treinamento e a avaliação em cada dobra e retornando os scores.

```
errado, certo = [], []
for semente in range(20):
    rng = np.random.default_rng(semente)
    X_ruido = rng.normal(size=(60, 2000))
    y_ruido = rng.integers(0, 2, 60)
    dobras = StratifiedKFold(5, shuffle=True, random_state=SEMENTE)

    # usa TODAS as linhas
    X_sel = SelectKBest(f_classif, k=20).fit_transform(X_ruido, y_ruido)
    errado.append(cross_val_score(LogisticRegression(max_iter=1000), X_sel,
                                   y_ruido, cv=dobras).mean())

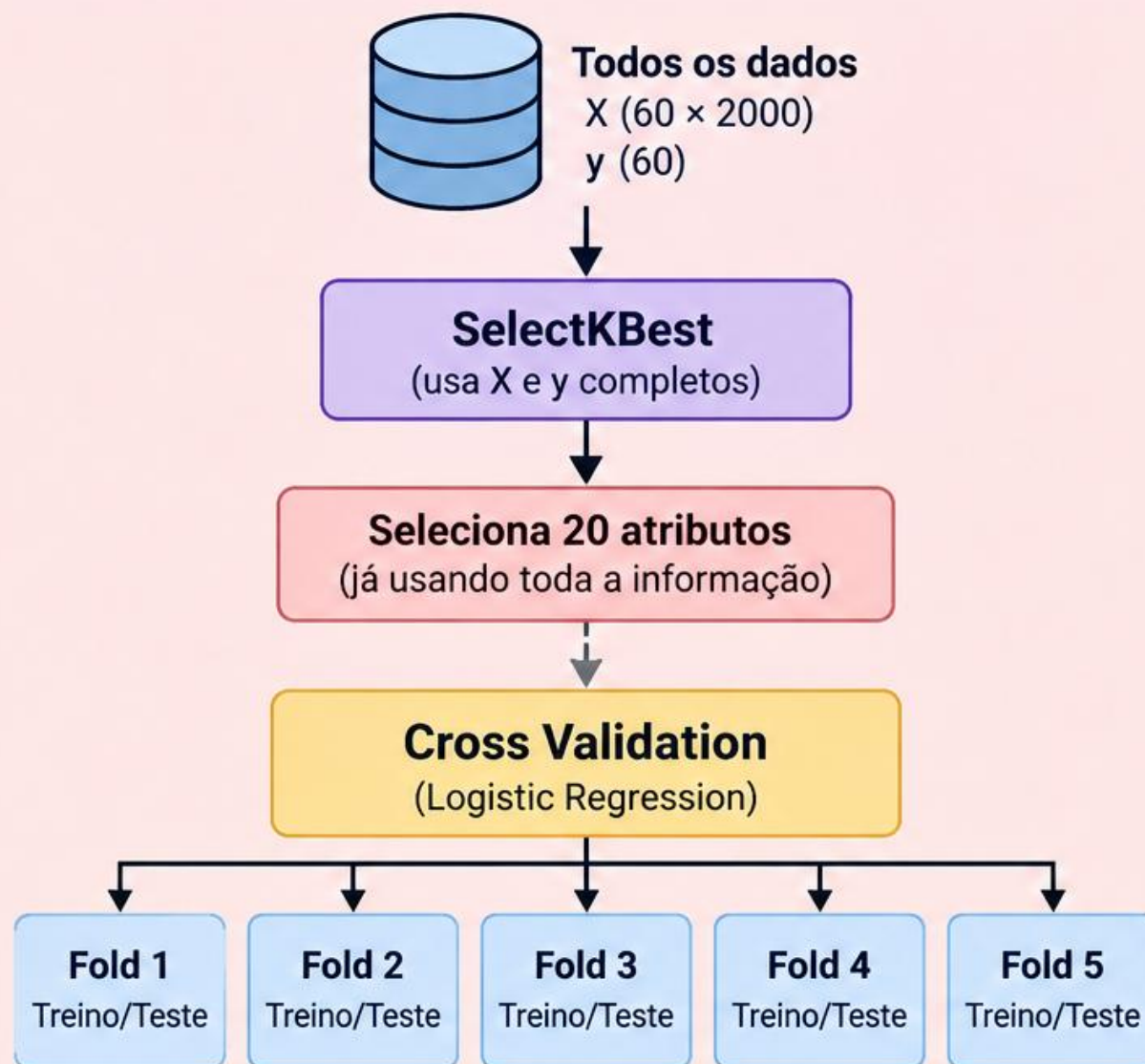
    # Maneira correta
    pipe = Pipeline([("sel", SelectKBest(f_classif, k=20)), ("modelo",
                                                              LogisticRegression(max_iter=1000))])
    certo.append(cross_val_score(pipe, X_ruido, y_ruido, cv=dobras).mean())
```



ERRADO

Vazamento de dados
(data leakage)

A seleção de atributos usa TODOS os dados (treino + validação)



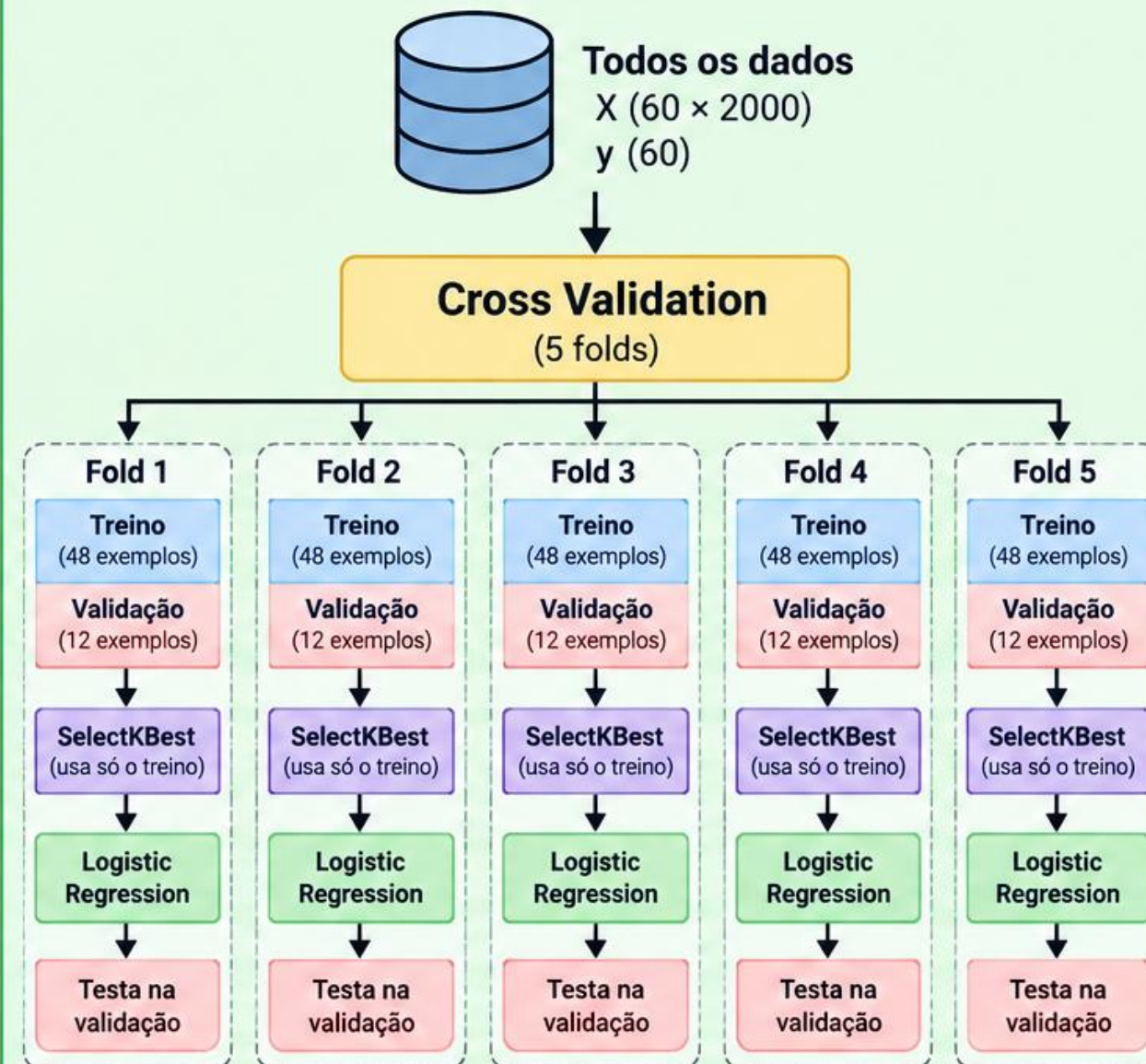
Problema: a seleção de atributos já viu os exemplos de validação, o que faz a avaliação ficar otimista (resultados artificialmente altos).



CERTO

Pipeline dentro da
validação cruzada

A seleção de atributos é ajustada apenas com os dados de treino de cada fold.



Correto: em cada fold, a seleção de atributos é feita apenas com os dados de treino. A validação fica independente e o resultado representa melhor a capacidade de generalização do modelo.

1

K-Fold Simples

Divide sem olhar as classes.

Quando usar?

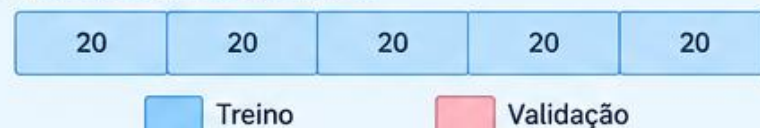
- Problemas de regressão (variável contínua).
- Classificação com classes equilibradas.
- Situações em que a distribuição das classes não é um problema.

Como funciona?

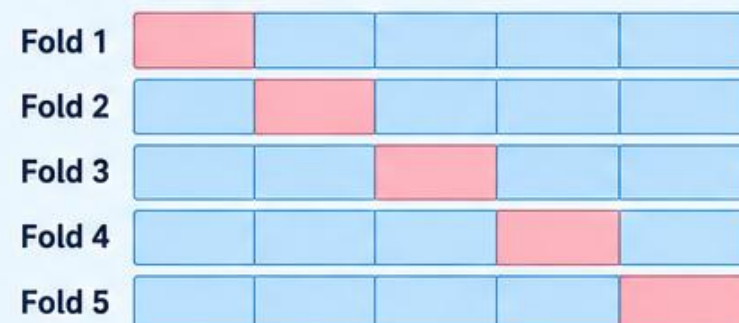
Divide o conjunto de dados em K partes (folds) de tamanho aproximadamente igual. Em cada rodada, uma parte é usada para validação e as demais para treino.

Exemplo: 100 registros, K = 5

Divisão do conjunto de dados:



Rodadas de treinamento e validação:



✓ Vantagens

- ✓ Simples e amplamente utilizado.
- ✓ Funciona bem para a maioria dos problemas.
- ✓ Boa estimativa do desempenho.

✗ Desvantagens

- ✗ Pode gerar folds desbalanceados em problemas de classificação com classes raras.

2

K-Fold Estratificado

Cada dobra preserva a proporção das classes.

Quando usar?

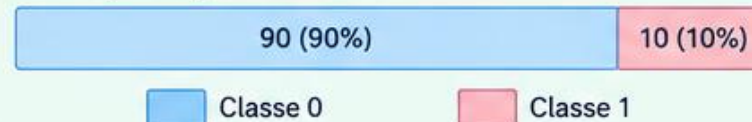
- Problemas de classificação.
- Quando as classes são desbalanceadas (classes raras).
- Quando queremos que cada fold tenha proporções semelhantes às do conjunto original.

Como funciona?

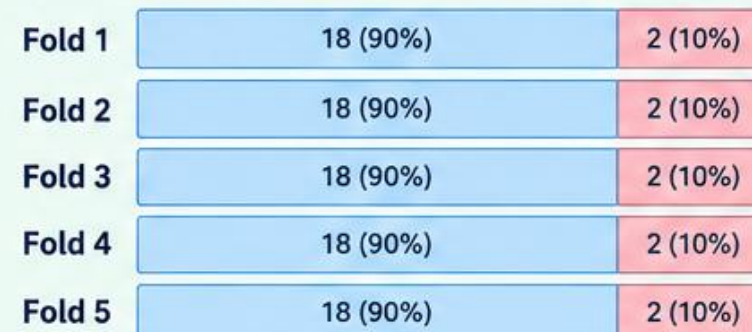
Divide o conjunto de dados em K partes, mantendo (o mais próximo possível) a proporção de cada classe em todas as dobras.

Exemplo: 100 registros (90 classe 0, 10 classe 1), K = 5

Distribuição original das classes:



Cada fold mantém aproximadamente a mesma proporção:



✓ Vantagens

- ✓ Mantém a proporção das classes em cada fold.
- ✓ Mais estável e confiável para conjuntos desbalanceados.
- ✓ Evita folds sem exemplos da classe rara.

✗ Desvantagens

- ✗ Normalmente usado apenas para classificação. Não se aplica diretamente a problemas de regressão.

3

Leave-One-Out (LOO)

Usa um exemplo para validação (K = N).

Quando usar?

- Bases de dados muito pequenas.
- Quando queremos aproveitar ao máximo os dados para treinamento.
- Situações em que o custo computacional não é um problema (grandes algoritmos podem ficar lentos).

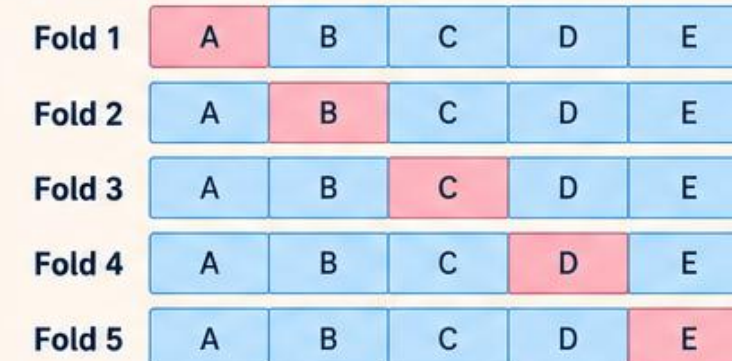
Como funciona?

É um caso especial do K-Fold em que K é igual ao número de exemplos (N). Em cada rodada, um único exemplo é usado para validação e os demais para treino.

Exemplo: 5 registros (K = 5)



Rodadas de treinamento e validação:



✓ Vantagens

- ✓ Aproveita quase todos os dados para treinamento (N - 1).
- ✓ Útil quando a base é muito pequena.
- ✓ Não depende de uma divisão aleatória.

✗ Desvantagens

- ✗ Alto custo computacional (treina N vezes). Pode ter maior variância na estimativa. Não é prático para bases grandes.

Todas essas técnicas são estratégias de validação cruzada, usadas para estimar o desempenho do modelo em dados não vistos.

**Regressão /
Classes Equilibradas**

K-Fold Simple (Divide sem
olhar as classes).

**Classificação /
Classes Raras**

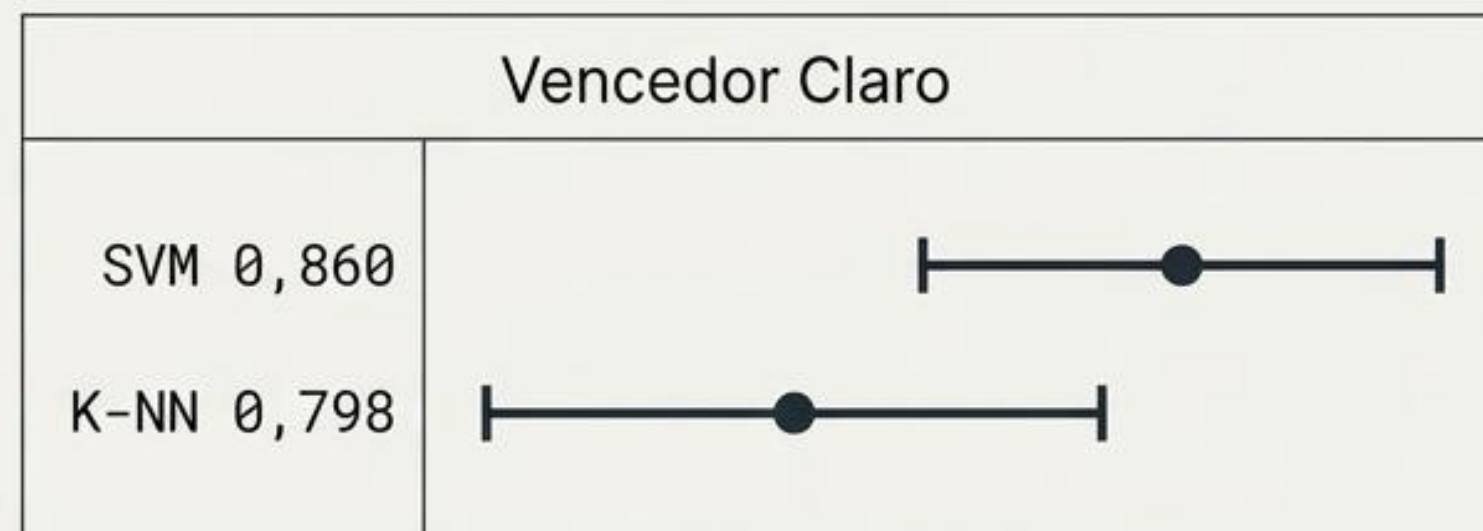
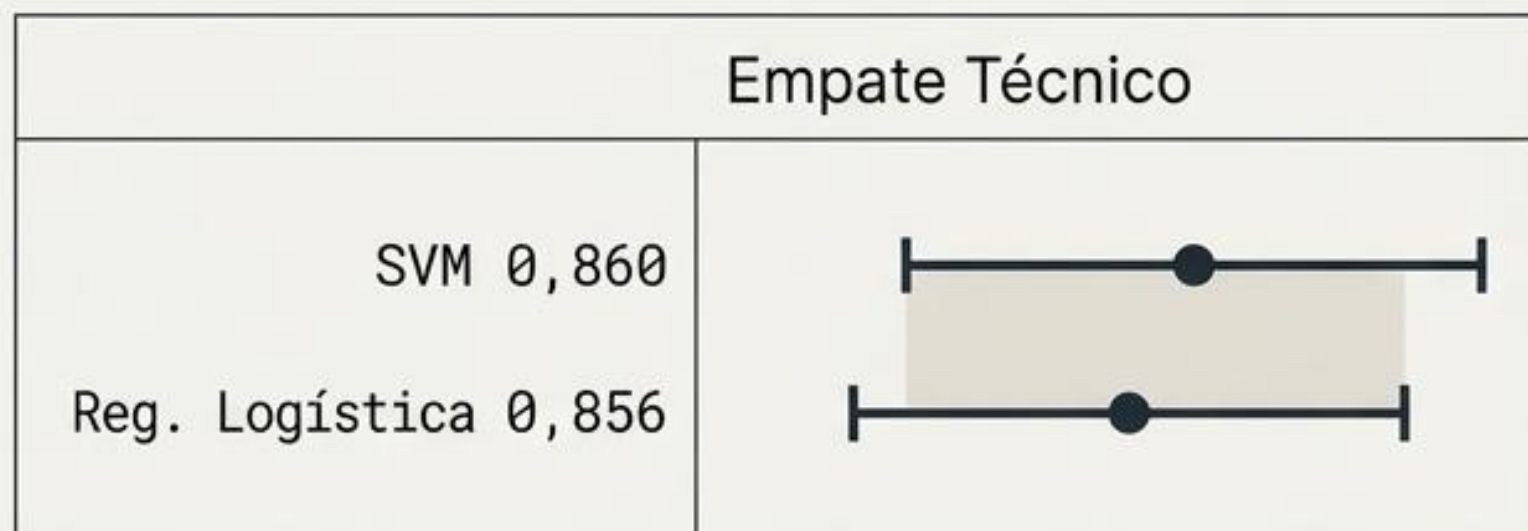
K-Fold Estratificado (O Padrão
Ouro. Cada dobra preserva a
proporção exata de cada classe).

Bases Minúsculas

Leave-one-out (K = número de
exemplos. Alto custo
computacional, alta variância).

Dica de Ouro: Sempre use `shuffle=True` para evitar vieses introduzidos pela ordem original das linhas no arquivo.

Empate Técnico em Machine Learning



Crucial Rule: Se a diferença entre as médias de dois modelos for menor que a variação (desvio padrão) entre as dobras, não declare um vencedor. Eles estão empatados.

Para uma comparação justa, todos os modelos devem ser avaliados exatamente nas mesmas dobras de dados.



Regra importante na comparação de modelos

Se a diferença entre as médias de dois modelos for menor que a variação (desvio padrão) entre as dobras, não declare um vencedor. Eles estão empatados.

Exemplo 1 – Diferença pequena (modelos empatados)

Modelo A (KNN)

Scores nas 5 dobras:
[0.82, 0.86, 0.84, 0.88, 0.85]

Média = 0.850
Desvio padrão = 0.020

Variou pouco entre as dobras
(modelo consistente).

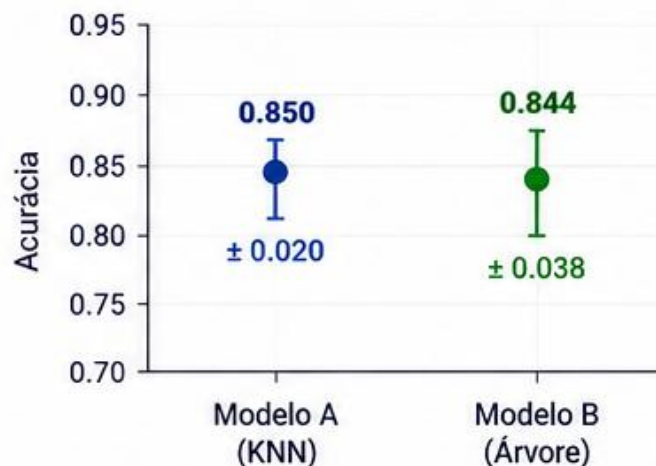
Modelo B (Árvore de Decisão)

Scores nas 5 dobras:
[0.81, 0.90, 0.80, 0.87, 0.84]

Média = 0.844
Desvio padrão = 0.038

Variou mais entre as dobras
(modelo menos estável).

Média e desvio padrão (barras de erro)



Análise

- A diferença entre as médias é:
 $0.850 - 0.844 = 0.006$ (0,6%)
- Essa diferença é menor que a variação observada entre as dobras (desvios de 0.020 e 0.038).
- Os intervalos de variação se sobrepõem.

Conclusão: não há evidência suficiente para dizer que um modelo é melhor. Eles estão empatados.

Exemplo 2 – Diferença grande (modelo claramente melhor)

Modelo A (Regressão Logística)

Scores nas 5 dobras:
[0.78, 0.80, 0.77, 0.79, 0.81]

Média = 0.790
Desvio padrão = 0.015

Desempenho estável.

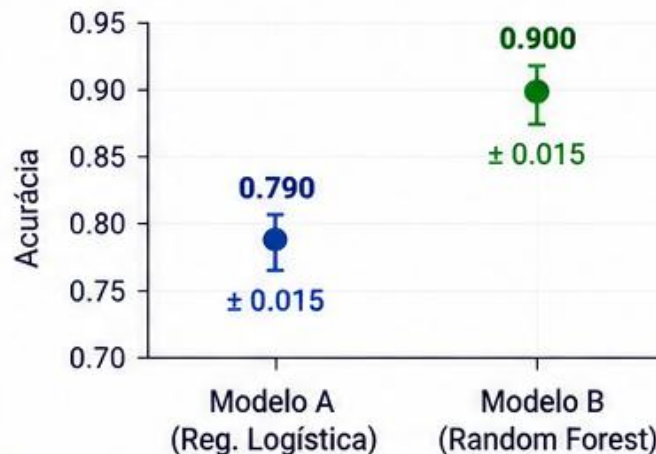
Modelo B (Random Forest)

Scores nas 5 dobras:
[0.88, 0.90, 0.89, 0.91, 0.92]

Média = 0.900
Desvio padrão = 0.015

Desempenho alto e estável.

Média e desvio padrão (barras de erro)



Análise

- A diferença entre as médias é:
 $0.900 - 0.790 = 0.110$ (11,0%)
- Essa diferença é muito maior que a variação entre as dobras (0.015).
- Os intervalos de variação não se sobrepõem.

Conclusão: o Modelo B tem desempenho claramente superior.

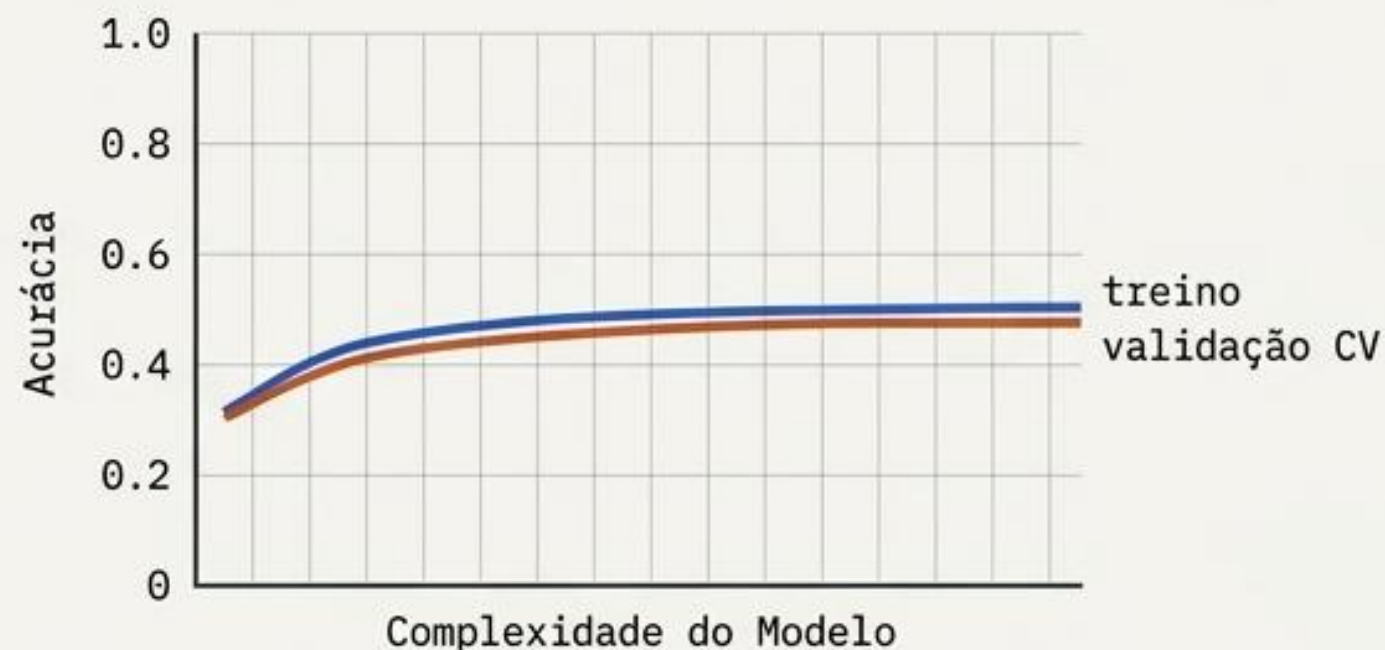


Portanto, ao comparar modelos com validação cruzada, observe a média e a variação (desvio padrão).

Se a diferença entre as médias for pequena em relação à variação, considere que os modelos estão empatados.

Diagnóstico: Complexidade do Modelo

Subajuste / Underfitting



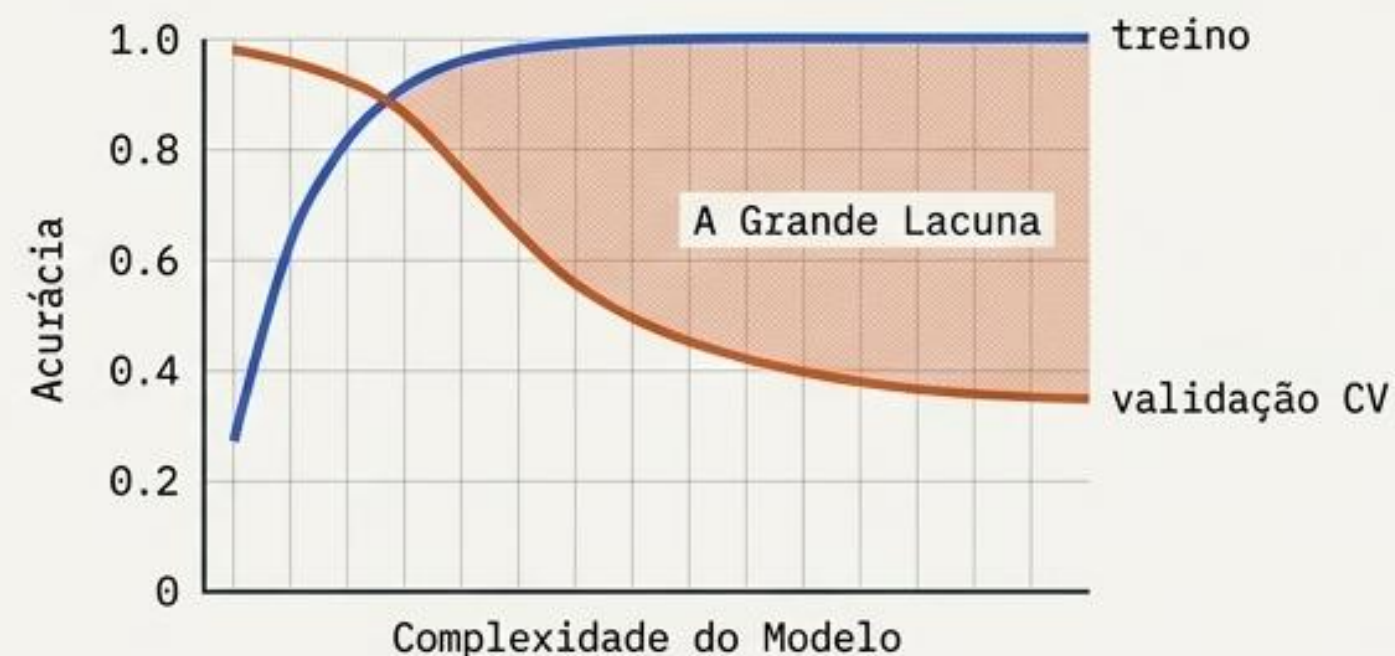
Sintoma:

Erro alto em ambos. O modelo é simples demais (ex: reta para dados curvos). Erra até o que já viu.

Cura:

Modelos mais flexíveis e complexos, adicionar mais atributos.

Sobreaajuste / Overfitting



Sintoma:

Treino perfeito, Validação péssima. A temida "Grande Lacuna". O modelo decorou ruído.

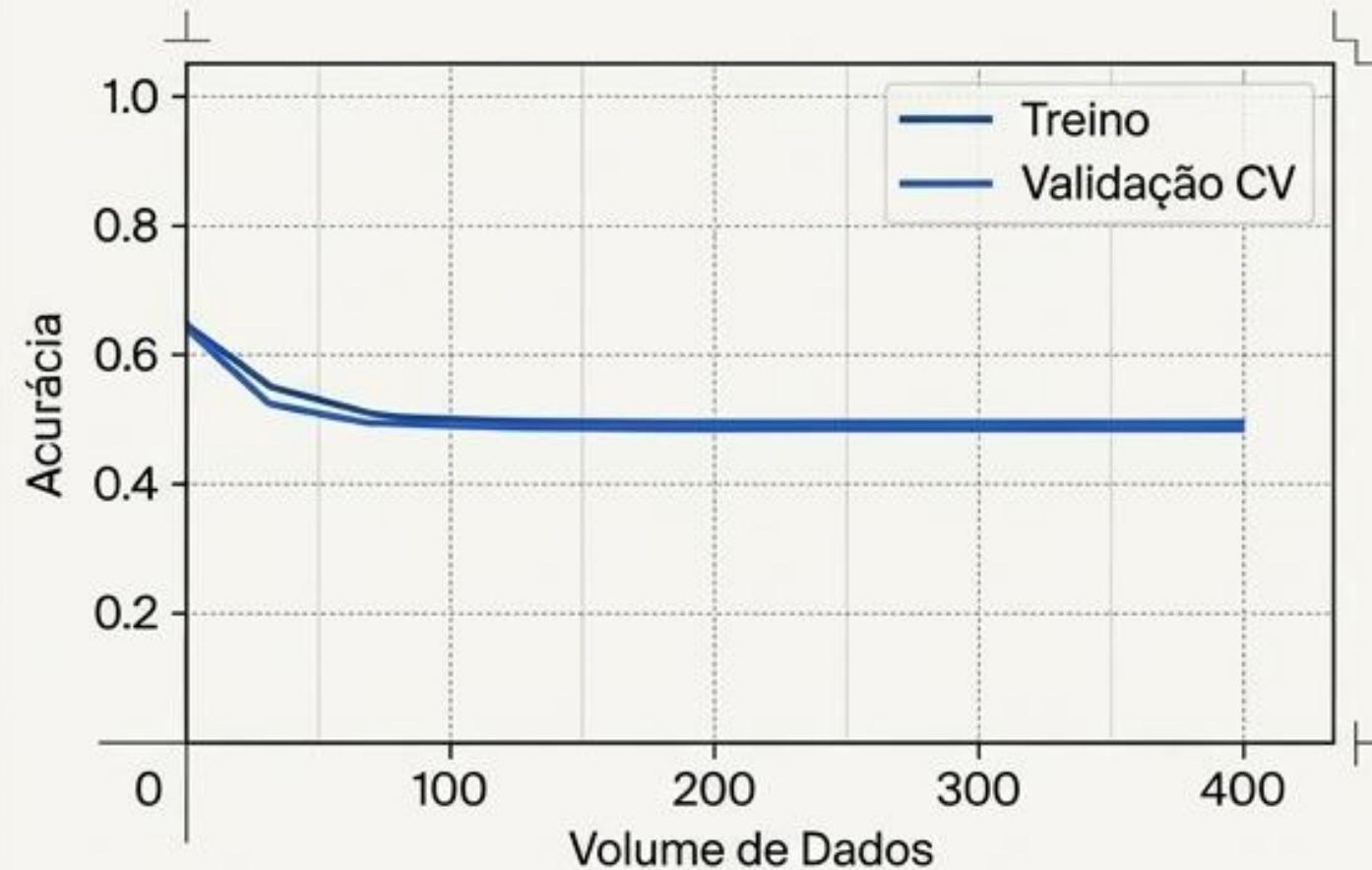
Cura:

Simplificar o modelo, regularizar, injetar mais dados.

Regra de Ouro: Escolha hiperparâmetros sempre pela curva de validação. No treino, o modelo mais complexo sempre parece o melhor.

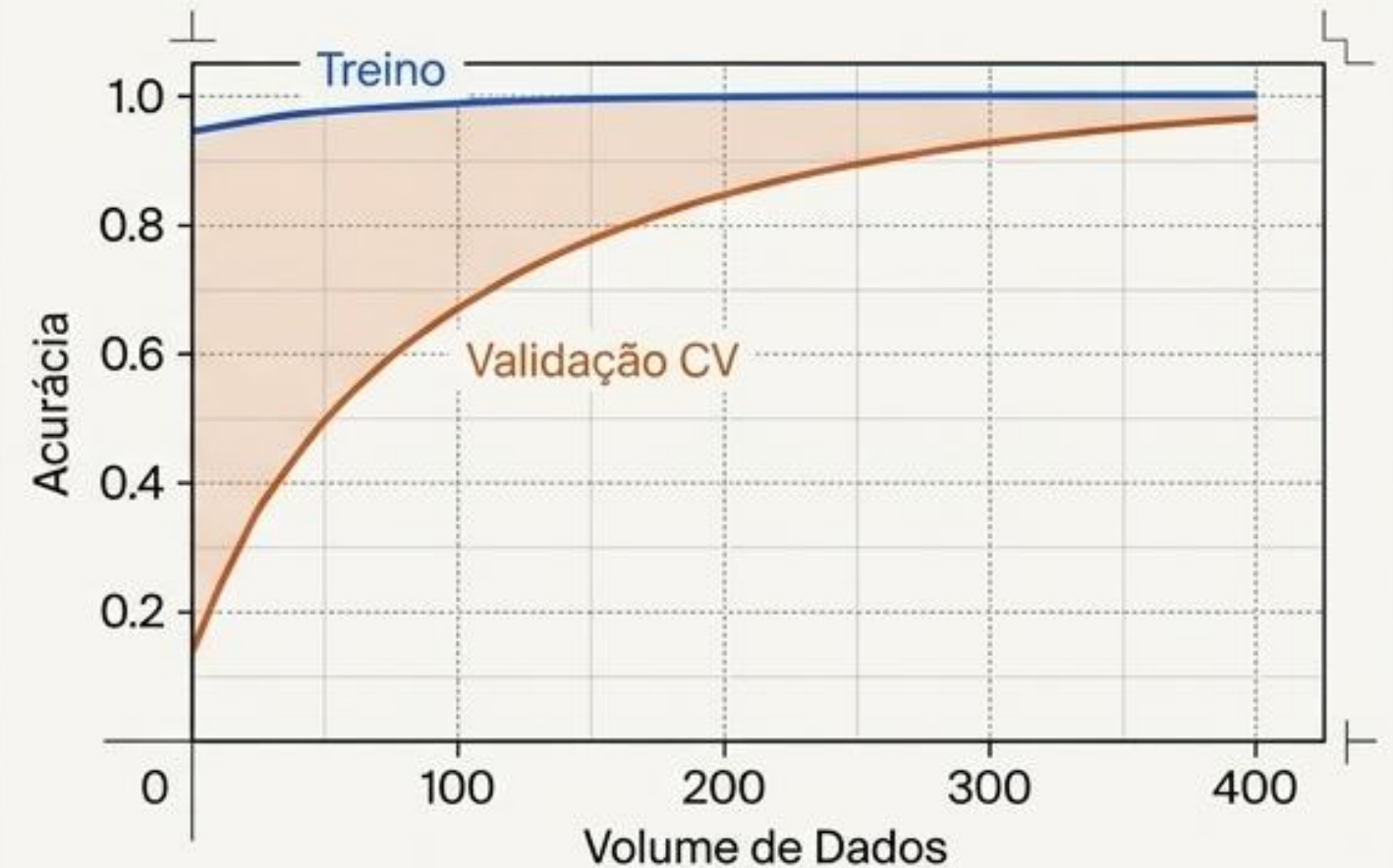
Curvas de Aprendizado: Injetar mais dados resolve?

Padrão 1 - Subajuste



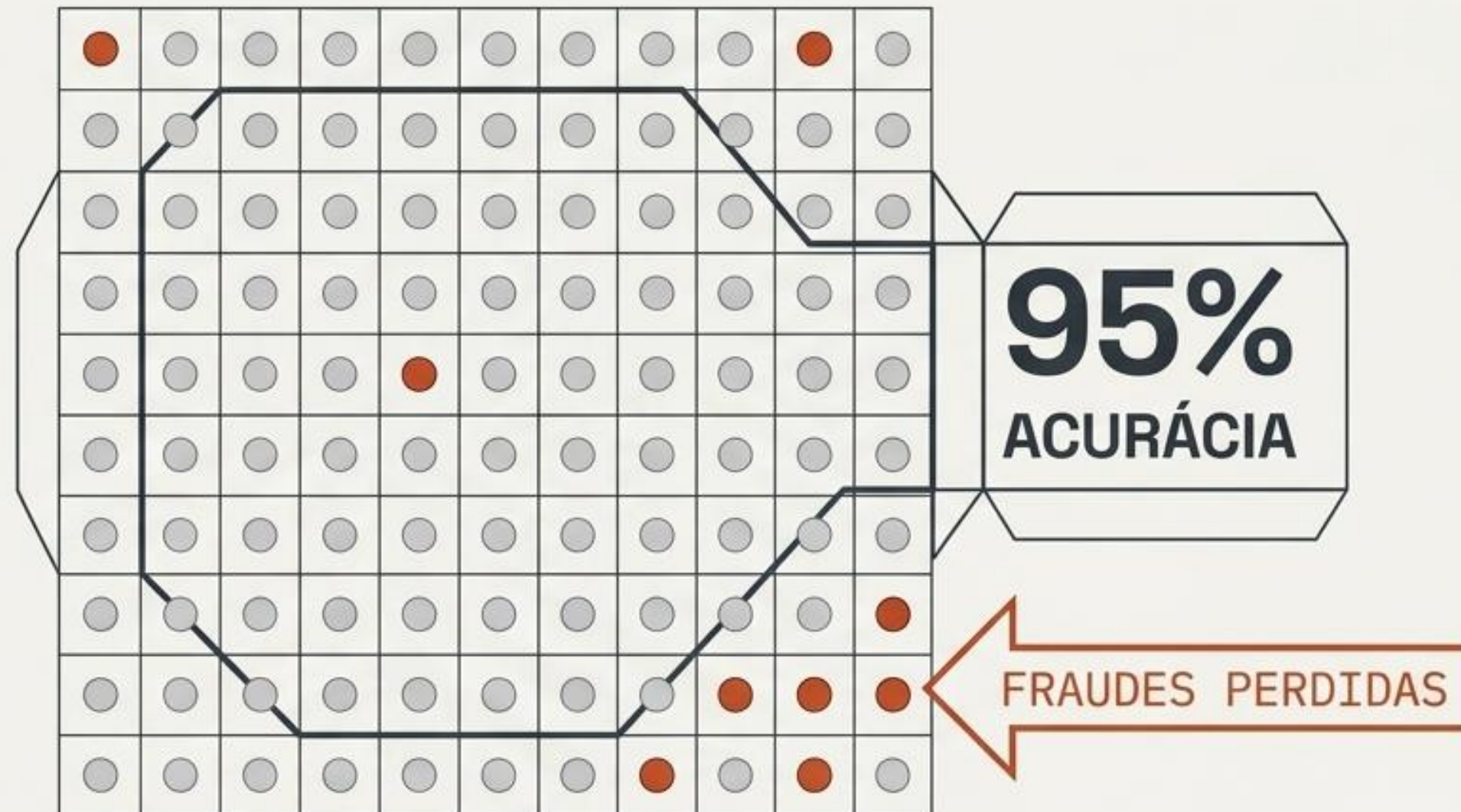
Curvas convergem rápido em um valor baixo. O limite está no modelo, não nos dados. Mais exemplos não ajudarão.

Padrão 2 - Sobreajuste



Grande lacuna entre treino (1.0) e validação, mas se aproximando. O modelo sofre de variância. Injetar mais dados é a cura provável.

O Paradoxo da Acurácia: Quando 95% de acerto é um desastre.



Em classes desbalanceadas (ex: fraudes são 5%), a acurácia é uma ilusão. Um modelo "preguiçoso" que chuta que "nunca há fraude" acerta 95% do tempo, mas deixa passar 100% dos crimes. Ele não aprendeu nada e é inútil para o negócio.

O Radar de Métricas: Além da Acurácia

Precisão



Dos que eu marquei como alvo, quantos eram **reais**?

Priorize quando o **falso alarme** é **caro** (ex: aprovar crédito ruim, bloquear um cliente honesto).

Recall



Dos alvos reais que existiam, quantos eu consegui **achar**?

Priorize quando **perder o alvo** é **fatal** (ex: deixar passar uma doença grave ou uma fraude).

F1-Score

O ponto de equilíbrio harmônico. Use quando a classe é rara e os dois erros doem no bolso.

O Radar de Métricas: Além da Acurácia

Diferentes métricas respondem a perguntas diferentes. Escolha a métrica certa para o seu problema.

Nem todo acerto tem o mesmo peso!



Precisão

Dos que eu marquei como alvo, quantos eram **reais**?

$$\text{Precisão} = \frac{VP}{VP + FP}$$

Exemplo: Detecção de fraudes bancárias

Das 10 transações que o modelo marcou como fraude, 8 realmente eram fraudes.

$$\text{Precisão} = 8 / (8 + 2) = 0,80 \text{ (80\%)}$$



Use quando o falso alarme é caro.

Ex.: bloquear um cliente honesto.



Recall

Dos alvos reais que existiam, quantos eu consegui **achar**?

$$\text{Recall} = \frac{VP}{VP + FN}$$

Exemplo: Diagnóstico médico

De 10 pacientes que realmente tinham a doença, o modelo encontrou 8.

$$\text{Recall} = 8 / (8 + 2) = 0,80 \text{ (80\%)}$$



Use quando perder o alvo é fatal.

Ex.: deixar passar uma doença grave.



F1-Score

O ponto de equilíbrio harmônico entre precisão e recall.

$$F1 = 2 \times \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}}$$

Exemplo: Base desbalanceada

Precisão = 0,80 Recall = 0,80

$$F1 = 2 \times \frac{0,80 \times 0,80}{0,80 + 0,80} = 0,80 \text{ (80\%)}$$



Use quando a classe é rara e os dois erros (FP e FN) são importantes.

Matriz de Confusão

Base para o cálculo das métricas

		Valor Real	
		Positivo (ex.: fraude)	Negativo (ex.: normal)
Previsão do Modelo	Positivo	VP (Verdadeiro Positivo) 8	FP (Falso Positivo) 2
	Negativo	FN (Falso Negativo) 2	VN (Verdadeiro Negativo) 88

Exemplo completo

Em 100 transações, o modelo gerou:

- 8 fraudes detectadas corretamente (VP)
- 2 falsos alarmes (FP)
- 2 fraudes que passaram (FN)
- 88 transações normais corretas (VN)

Métricas calculadas:

- Precisão = $8 / (8 + 2) = 0,80$ (80%)
- Recall = $8 / (8 + 2) = 0,80$ (80%)
- F1-Score = **0,80 (80%)**
- Acurácia = $(8 + 88) / 100 = 0,96$ (96%)



Por que olhar além da acurácia?

Em bases desbalanceadas, a acurácia pode enganar.

Exemplo: 1000 pacientes (10 com a doença, 990 saudáveis)
Modelo que diz "todos saudáveis":

- Acurácia = $990 / 1000 = 99\%$ (parece excelente)
- Recall = 0% (não encontrou nenhum doente)
- Precisão = indefinida (nenhum positivo previsto)

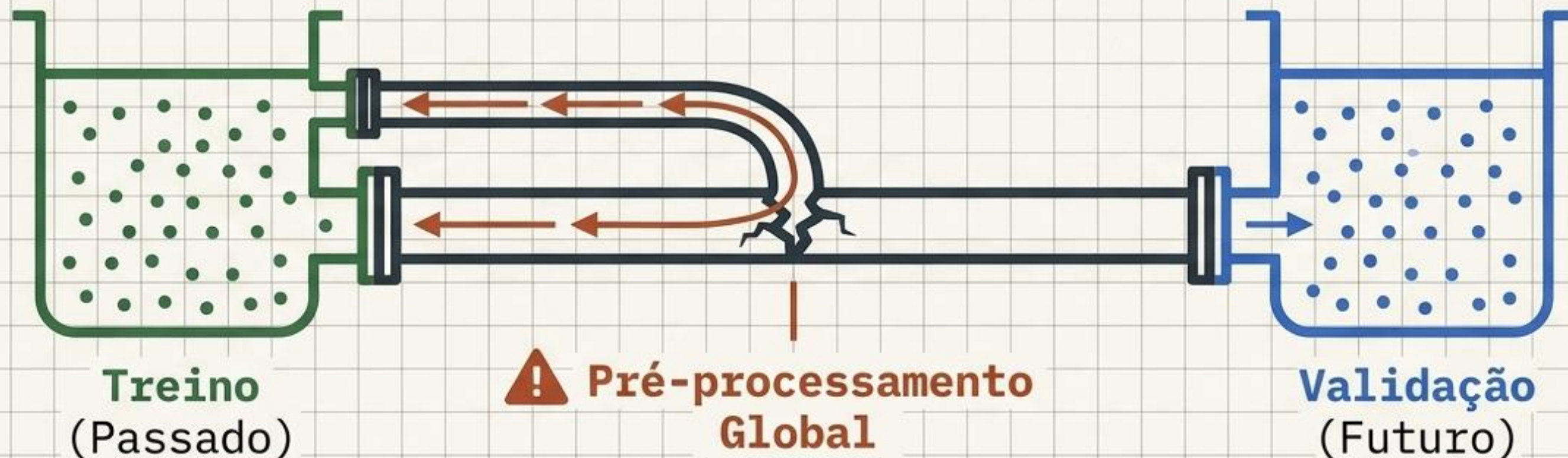
Alta acurácia nem sempre significa um bom modelo. Analise precisão, recall e F1, especialmente em problemas desbalanceados.



Cada métrica é um ângulo de visão. Use o conjunto delas para tomar a melhor decisão.

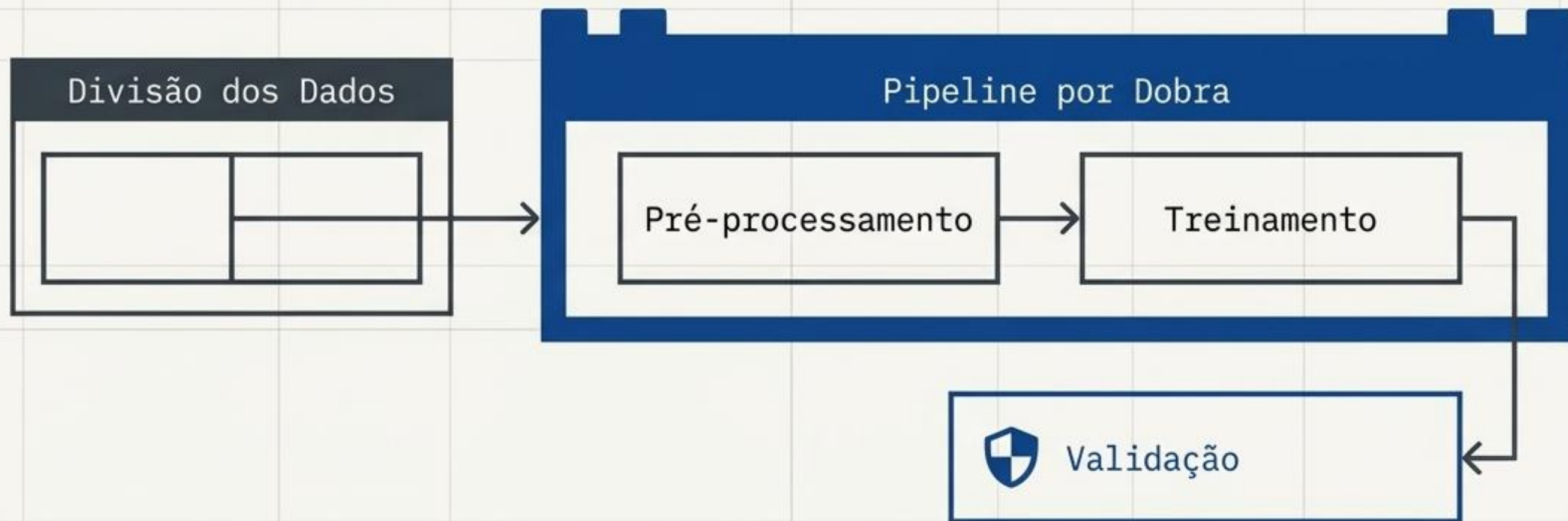
Métricas guiam melhores decisões!

Vazamento de Dados (A Ilusão Mágica)



O que é	Causas Comuns
Quando o modelo "vê a resposta" antes do teste. A acurácia sobe magicamente de 50% para mais de 90% em dados de puro ruído. O modelo previu o passado.	Padronizar variáveis, imputar valores nulos ou selecionar atributos usando o dataset inteiro ANTES de dividir as dobras. A validação já contaminou a matemática do treino.

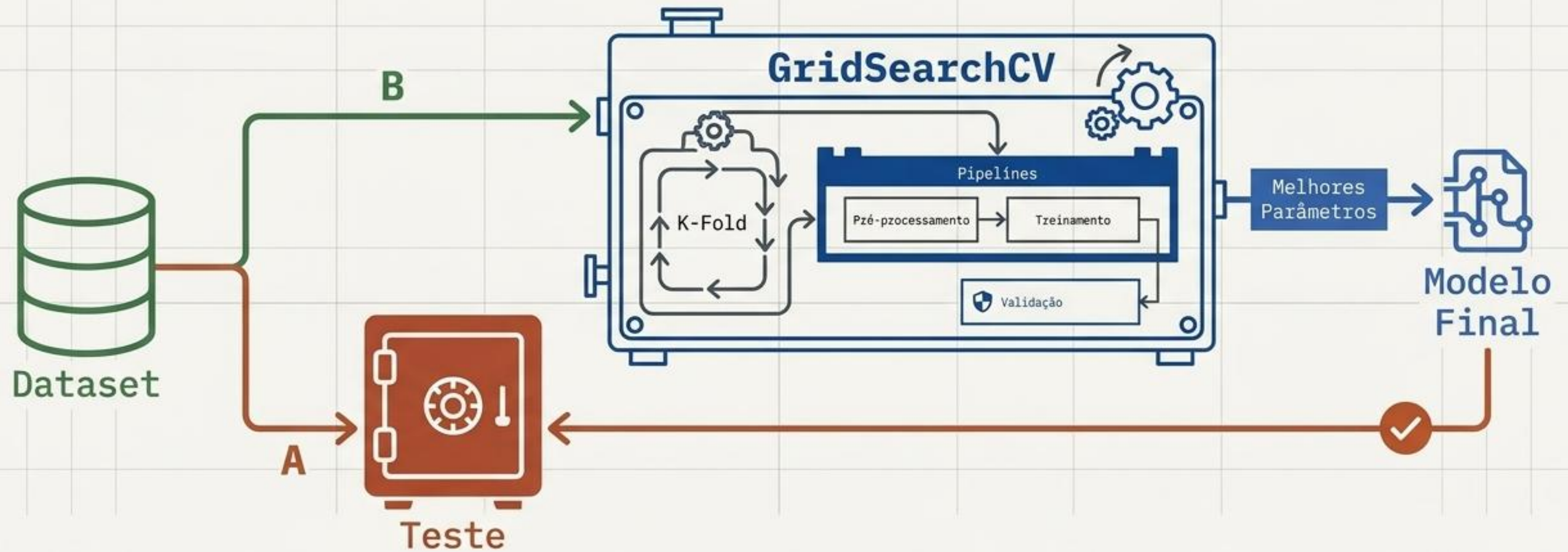
0 Escudo: Protegendo o modelo com Pipelines



A Solução: Qualquer passo que “aprenda” algo dos dados deve ser ajustado do zero apenas com o treino de cada dobra.

0 Código: `pipe = Pipeline([('prep', preprocessador), ('modelo', Algoritmo())])`.
O pipeline garante automaticamente que o pré-processamento jamais veja os dados de validação durante o Cross-Validation.

A Arquitetura Suprema: GridSearchCV



A Busca Honesta:

Automatiza o encontro do melhor hiperparâmetro sem trair o teste.

Regra Respeitada:

A validação cruzada acontece exclusivamente dentro dos dados de treino.
O teste final permanece imaculado.

0 Checklist do Auditor

- [✓] Há um baseline simples (chute da classe mais comum) para comparar?
- [✓] O conjunto de Teste ficou em um cofre e foi usado uma única vez no fim?
- [✓] Todo o pré-processamento ocorreu isolado dentro de um Pipeline?
- [✓] A avaliação usou K-Fold Estratificado e reportou Média $\pm$ Desvio Padrão?
- [✓] Modelos empatados (diferença menor que o desvio) foram tratados como iguais?
- [✓] A métrica escolhida (Precisão/Recall/F1) reflete a dor de negócio da classe rara?
- [✓] As curvas de complexidade e aprendizado foram analisadas contra sub/sobreaajuste?